

Silicon Photonics PIC Design & Automation Platform - Detailed Curriculum

Yes. Before starting Chapter 02, this is a good time to lock down the complete curriculum. The goal is not simply “learn GDSFactory.” The end goal is a portfolio-quality **silicon-photonics design-automation project** that demonstrates Python, GDSFactory, PIC design, EM simulation, parameter sweeps, hierarchical layout, verification, and fabrication-oriented workflow.

Given the depth we are aiming for, I would keep the program at **8 chapters**.

Overall project

Silicon Photonics PIC Design & Automation Platform

By the end, you should be able to go from:

```
Device specifications
  ↓
Python parameters
  ↓
Parameterized photonic components
  ↓
EM / analytical simulation
  ↓
Automated optimization / sweeps
  ↓
Circuit assembly
  ↓
Automatic routing
  ↓
Photonic test-chip generation
  ↓
DRC / connectivity verification
  ↓
GDS
  ↓
Fabrication-oriented design package
```

How every chapter will work

For every significant concept, we'll use essentially the learning loop you've asked for:

```
1. Explain concept
  ↓
2. Guided example
  ↓
3. Small assignment
  ↓
```

4. You solve it
- ↓
5. I review/debug it
- ↓
6. Harder assignment
- ↓
7. Chapter mini-project
- ↓
8. Clean files
- ↓
9. Git checkpoint

I will increasingly stop giving you complete code as you progress.

By Chapters 5-8, assignments should look more like engineering specifications rather than coding exercises.

Chapter 01 — Python & GDSFactory Fundamentals

Status: ~technically complete

The purpose was to learn the GDSFactory mental model and enough Python to work productively.

You learned:

Step	Concept
1	Python/GDSFactory environment
2	Generic PDK activation
3	<code>gf.Component()</code>
4	Standard GDSFactory components
5	<code>add_ref()</code>
6	Component vs reference
7	Optical ports
8	<code>.connect()</code>
9	<code>.move()</code>
10	Automatic routing
11	Cross-sections
12	Bend-radius constraints
13	Parent-level ports
14	Parameterized Python functions
15	<code>@gf.cell</code>
16	Cell caching
17	Hierarchical layout
18	Top-level components
19	Bounding boxes
20	GDS export
21	KLayout inspection

Your final mini-project was the two-channel hierarchical photonic layout.

Chapter 01 finishes when

We complete the housekeeping that we temporarily paused:

- ✓ working mini-project
- ✓ organized files
- `.gitignore`

- clean Git staging
- Chapter 01 commit
- push to GitHub

Suggested final commit:

Complete Chapter 01 GDSFactory fundamentals

After that, we freeze Chapter 01 and move forward.

Chapter 02 — Components, Ports & Reusable Building Blocks

This chapter transitions from learning GDSFactory syntax to constructing an actual **photonic component library**.

Part 2.1 — More serious port handling

We'll go deeper into:

```
port name
port center
port width
port orientation
port type
port layer
```

You will inspect and manipulate ports rather than merely connecting o1 and o2.

Assignment example:

Given four ports with different orientations, determine which components can connect directly and which require transformations.

Part 2.2 — Translation, rotation, and mirroring

You'll learn how references are transformed:

```
move
rotate
mirror
```

and how their ports transform automatically.

For example:

original

o1 ——— o2

rotate 90°

```

  o2
  |
  o1
```

Assignment:

Construct four copies of one component at four orientations without creating four separate component definitions.

Part 2.3 — Cross-sections properly

We used "strip" casually in Chapter 01.

Now we'll properly understand a cross-section as a technology definition containing things such as:

- waveguide width
- layer
- offsets
- cladding
- routing behavior
- bend constraints

You will build and inspect custom cross-sections.

Assignment:

Generate 450 nm, 500 nm, and 600 nm waveguide variants from one component generator.

Part 2.4 — Tapers

We will build:



and understand why transitions are needed physically.

You'll parameterize:

- input width
- output width
- length

Part 2.5 — Splitters and combiners

We'll explore existing GDSFactory structures such as:

- Y branches
- MMIs
- directional couplers

You'll inspect their ports and hierarchy.

Part 2.6 — Reusable custom components

You will start organizing code into actual reusable modules instead of one Python file per experiment.

Eventually:

```
components/  
├── waveguides.py  
├── tapers.py  
├── couplers.py  
└── splitters.py
```

Chapter 02 mini-project

Build a reusable **optical building-block library** containing at least:

- straight waveguide
- 90° bend
- taper
- splitter

coupler
custom routed section

Each must:

- be parameterized
- expose meaningful ports
- use `@gf.cell`
- support hierarchy
- generate valid GDS

Chapter 02 finishes with

Reusable component library
+
component GDS gallery
+
small test script
+
README documentation
+
Git checkpoint

Expected commit:

Add reusable photonic component library

Chapter 03 — Routing & PIC Circuit Assembly

Chapter 02 gives us components.

Chapter 03 teaches us to build **circuits**.

3.1 Placement versus routing

We'll formalize:

placement = where components are located

routing = how their ports are connected

This is analogous to electronic IC physical design.

3.2 Single optical routing

We'll go much deeper into:

`route_single()`

including:

cross-section

bend radius

port orientations

route length

geometry constraints

3.3 Route bundles

Instead of:

one input → one output

we'll route arrays such as:

```
I1 ——— 01
I2 ——— 02
I3 ——— 03
I4 ——— 04
```

using bundle routing.

This is essential for test chips.

3.4 Routing conflicts

We'll intentionally create bad floorplans and see:

route crossing
impossible bend radius
insufficient spacing
bad port orientation

Then correct the floorplan.

3.5 Path-length control

This becomes important for interferometers.

For example:

arm 1 = L

arm 2 = L + ΔL

You'll calculate and control optical path differences.

3.6 Circuit hierarchy

We'll build:

```
chip
├── circuit
│   ├── input block
│   ├── device
│   └── output block
```

instead of putting everything into one giant component.

3.7 Automated device placement

Instead of manually writing:

```
ref1.move(...)
ref2.move(...)
ref3.move(...)
```

we'll start using loops and design rules.

For example:

device 1 → x=0
device 2 → x=pitch
device 3 → x=2*pitch
...

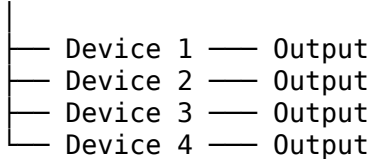
This is where Python starts giving us serious leverage.

Chapter 03 mini-project

Construct a multi-channel routed PIC containing several reusable components.

Something conceptually like:

Input array



with automatic placement and routing.

Chapter 03 finishes with

Parameterized multi-channel circuit

+
automatic placement
+
automatic routing
+
top-level ports
+
clean hierarchy
+
GDS
+
Git checkpoint

Chapter 04 — Real Silicon-Photonics Devices

This is where the project becomes much more photonics-heavy.

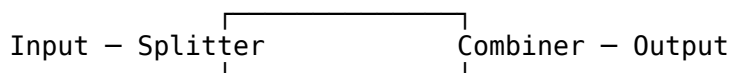
We'll build devices that recruiters actually recognize immediately.

4.1 Mach-Zehnder interferometer

We'll revisit the physics:

$$\Delta\phi = (2\pi/\lambda)n_{\text{eff}}\Delta L$$

and then build:



Parameters will include:

arm length
 ΔL
bend radius

waveguide width
splitter type

4.2 Ring resonator

We'll build:

———— bus waveguide
 ○ ring

Parameters:

radius
gap
waveguide width
coupling region

We'll connect geometry to concepts such as:

FSR
Q
coupling
resonance

4.3 Directional coupler

We'll investigate:

gap
interaction length
coupling ratio
and later simulate it.

4.4 Bragg grating

Parameterized:

period
number of periods
duty cycle
corrugation width

You'll generate hundreds of periods automatically.

This is an excellent demonstration of why Python beats manual CAD.

4.5 Device arrays

We'll generate:

ring R=5
ring R=6
ring R=7
ring R=8
...

automatically.

Same for:

Bragg periods
coupling gaps
waveguide widths
MZI ΔL

Chapter 04 mini-project

Create your own **silicon-photonic device library**:

MZI
ring resonator
directional coupler
Bragg grating
taper
routing structures

Each device gets:

Python generator
parameters
ports
GDS
brief physical explanation

Chapter 04 finishes with

A documented device library suitable for reuse in the final chip.

This is also where your GitHub repository should start becoming genuinely interesting to a photonics engineer.

Chapter 05 — Simulation & Automated Parameter Sweeps

This will probably be one of the largest chapters.

For your career objective, I recommend making **Lumerical the primary EM tool**, while using Python analytical models extensively and optionally reproducing selected examples with Meep.

The important skill is not simply clicking buttons in Lumerical.

It is:

Python-driven photonic simulation automation.

5.1 Waveguide mode analysis

Using MODE/FDE:

waveguide geometry
↓
mode solver
↓
neff
ng
mode profile
loss

We'll sweep:

waveguide width
wavelength
thickness where applicable

5.2 Single-mode analysis

You'll determine which widths support:

TE0
TE1
TM0
...

and produce automated plots.

5.3 Group index and dispersion

We'll calculate:

$$n_g = n_{\text{eff}} - \lambda (dn_{\text{eff}}/d\lambda)$$

and connect that to:

MZI FSR
ring FSR
dispersion

5.4 Bend simulations

We'll investigate:

radius → bend loss

This connects directly to what you learned in Chapter 01.

5.5 Directional-coupler simulation

Automated sweeps over:

gap
coupling length
wavelength

Output:

coupling ratio
transmission spectrum

5.6 Ring-resonator model

We'll start with a Python analytical model.

Then compare against EM-derived parameters.

We'll extract:

resonance wavelengths
FSR
Q
extinction

5.7 Bragg simulation

Sweep:

period
corrugation
number of periods
and generate spectra.

5.8 Lumerical Python API

This is important.

Eventually the workflow becomes:

```
Python
↓
create/update Lumerical model
↓
run simulation
↓
retrieve result
↓
store data
↓
plot
↓
choose optimum
```

Rather than manually performing each simulation.

5.9 Automated parameter sweeps

Example:

width:
450 → 550 nm

gap:
100 → 300 nm

wavelength:
1500 → 1600 nm

Python generates the design matrix and analyzes the results.

5.10 Data management

We'll stop using random arrays scattered through scripts.

Simulation outputs will be stored systematically:

```
results/
├── waveguide_sweep.csv
├── coupler_sweep.csv
├── ring_sweep.csv
└── plots/
```

5.11 Optimization

We'll eventually formulate problems such as:

Find the directional-coupler gap and length producing approximately 50/50 coupling at 1550 nm.

Python will search the parameter space.

Chapter 05 mini-project

Automate the design of one real photonic device from:

design target

↓

parameter sweep

↓

simulation

↓

analysis

↓

best candidate

↓

GDS generation

A directional coupler or ring resonator would be particularly suitable.

Chapter 05 finishes with

A working **simulation automation pipeline** rather than just isolated Lumerical screenshots.

Chapter 06 — Automated Photonic Test Chip

This is one of the central portfolio chapters.

Now all our devices become a real test chip.

6.1 Define the experimental matrix

For example:

Ring radius:

5, 10, 15 μm

Gap:

150, 200, 250 nm

Python creates all combinations automatically.

6.2 Waveguide test structures

We'll include structures such as:

straight reference

bend-loss structures

width sweeps

cutback structures

6.3 Ring arrays

Generate dozens of ring variations automatically.

6.4 MZI arrays

Sweep:

ΔL
waveguide width
bend radius

6.5 Bragg arrays

Sweep:

period
corrugation
number of periods

6.6 Optical I/O

We'll build consistent input/output architecture.

Depending on the design assumptions:

grating couplers
edge couplers
test ports

6.7 Device labels

The chip must be experimentally usable.

Instead of:

mystery device

we need labels such as:

RING_R10_G200
MZI_DL50
BRAGG_P315_D50

6.8 Automated device manifest

Python will generate something like:

ID	Device	Radius	Gap	X	Y
R01	Ring	10	200	100	500
R02	Ring	10	250	100	800

This connects the layout to experimental characterization.

6.9 Floorplanning

We'll define:

die dimensions
device pitch
I/O locations
routing regions
scribe margins

6.10 Alignment structures

We'll introduce:

- alignment marks
- measurement references
- die labels

6.11 Automated chip generation

Eventually:

```
generate_chip(parameters)
```

produces the entire test chip.

Chapter 06 final project

Generate a complete **Silicon Photonics Test Chip** containing:

- waveguide test structures
- MZIs
- rings
- Bragg gratings
- couplers
- parameter sweeps
- labels
- I/O
- routing
- manifest

Chapter 06 finishes with

- complete chip GDS
- KLayout screenshots
- device manifest CSV
- parameter table
- README
- architecture diagram
- Git checkpoint

This should already be a strong standalone GitHub portfolio artifact.

Chapter 07 — DRC, Connectivity & Fabrication-Oriented Design

Until now, a layout that “looks correct” has been enough.

From here:

It must be manufacturable.

7.1 What a real PDK contains

We'll distinguish:

- generic PDK
- vs
- foundry PDK

including:

- layers
- cross-sections
- minimum widths
- minimum gaps
- etch definitions
- implant layers
- metal layers
- design rules

7.2 Minimum feature checks

We'll intentionally make invalid structures:

- waveguide too narrow
- gap too small
- bend too tight

and detect/fix them.

7.3 KLayout DRC

We'll learn the concept of rule checking through KLayout.

Example rules:

- minimum width
- minimum spacing
- minimum enclosure

7.4 Connectivity

We will extract/check which optical ports are actually connected.

This is the photonic analogue of checking circuit nets.

7.5 Netlists

Our hierarchical PIC will be represented logically as well as geometrically.

For example:

MZI

- splitter
- upper arm
- lower arm
- combiner

7.6 LVS concepts

We'll understand:

- schematic/design intent
- vs
- layout connectivity

and what photonic LVS is trying to verify.

7.7 Cell naming and hierarchy hygiene

We'll detect:

- duplicate cell names
- incorrect references
- unintended flattening

7.8 Fabrication export

We'll prepare:

- GDS/OASIS
- layer information
- device manifest
- design notes
- DRC report

Chapter 07 mini-project

Take the Chapter 06 test chip and run it through a fabrication-oriented review.

Chapter 07 finishes with

- DRC-clean layout
- +
- connectivity verification
- +
- organized hierarchy
- +
- fabrication checklist
- +
- final GDS/OASIS

Even without access to a commercial foundry PDK, we can build a realistic mock fabrication-rule workflow.

Chapter 08 — Final Industry-Style Portfolio Project

Everything comes together here.

This should no longer feel like a course assignment.

You receive a **design specification** and build the system.

The project I recommend is:

Automated Silicon-Photonics Test-Chip Generator

The user provides something like:

wavelength: 1550

waveguide:
widths: [0.45, 0.50, 0.55]

rings:
radius: [5, 10, 15]
gap: [0.15, 0.20, 0.25]

```
mzi:
  delta_length: [20, 50, 100]
```

```
bragg:
  period: [0.31, 0.32, 0.33]
```

Then your software automatically produces:

```
device variants
↓
simulation cases
↓
optimized parameters
↓
layout cells
↓
device placement
↓
routing
↓
labels
↓
test-chip GDS
↓
manifest
↓
verification
```

Chapter 08.1 — Software architecture

We'll reorganize the project into something closer to:

PIC-silicon-photonics-automation/

```
— README.md
— configs/
— src/
  — pic_automation/
    — components/
    — circuits/
    — routing/
    — simulation/
    — chip/
— examples/
— tests/
— gds/
— results/
— docs/
```

This matters professionally.

8.2 Configuration-driven design

Design parameters should no longer require editing source code.

For example:

```
config.yaml
```

↓
Python
↓
chip

8.3 Component library integration

Reuse everything from earlier chapters.

No copy/pasting old designs.

8.4 Simulation integration

Key devices receive automated simulation or model-based evaluation.

8.5 Automated design selection

Example:

target = 50/50 coupler @1550 nm

simulation sweep

↓

best parameters

↓

layout

8.6 Automated chip assembly

The program determines:

which devices

how many

where placed

how routed

how labeled

8.7 Verification

Run:

design-rule checks

connectivity checks

parameter sanity checks

8.8 Automated reporting

Generate:

device table

simulation results

design parameters

GDS

plots

chip screenshots

8.9 Testing

We'll introduce basic software tests.

For example:

Does component contain expected ports?

Is width positive?

Is radius valid?

Does generated device count match specification?

This is important because design automation is software engineering as well as photonics.

8.10 Portfolio preparation

Finally we'll make the GitHub repository understandable to somebody who has never met you.

The README should answer:

What problem does this solve?

What technologies are used?

What does the architecture look like?

What photonic devices are supported?

How is simulation performed?

How is the chip generated?

What does the final layout look like?

How can someone reproduce it?

Final Chapter 08 deliverables

By the end, your repository should contain:

- ✓ Python/GDSFactory component library
- ✓ MZI generator
- ✓ ring generator
- ✓ Bragg generator
- ✓ coupler generator
- ✓ automated parameter sweeps
- ✓ Lumerical automation
- ✓ circuit routing
- ✓ photonic test-chip generator
- ✓ design manifest
- ✓ DRC/verification workflow
- ✓ final GDS
- ✓ simulation plots
- ✓ example configuration files
- ✓ documentation
- ✓ professional README
- ✓ clean Git history

And ideally one command should eventually be capable of doing something like:

```
python generate_chip.py --config configs/test_chip.yaml
```

and producing the chip.

That is where I want the project to end.

How difficulty will increase

The assignments will deliberately change character.

In Chapter 02, I may say:

Create a taper from 450 nm to 1 μm .

In Chapter 04:

Build an MZI with a 50 μm arm-length difference and expose two optical ports.

By Chapter 06:

Generate a 4×5 matrix of ring resonators sweeping radius and coupling gap. Label every device automatically and produce a CSV manifest.

And in Chapter 08 I may simply give you:

Design a test chip that experimentally characterizes waveguide width, ring radius, and coupling gap while fitting inside a specified die area.

At that point, **you decide the software architecture and implementation.**

Approximate scale of the whole program

At the ~5 hours/day pace you mentioned previously, I would treat this as roughly a **200-300 hour serious portfolio program**, not a weekend tutorial. Chapter 05 and the final capstone will probably consume the most time because simulation automation and integration are substantially harder than generating layouts.

A rough balance would be:

Chapter	Approx. effort
01 Fundamentals	10-15 h
02 Components	15-20 h
03 Routing/circuits	20-25 h
04 PIC devices	25-35 h
05 Simulation/optimization	40-60 h
06 Test chip	30-40 h
07 Verification/fabrication	20-30 h
08 Final capstone	40-60 h

The key point is that we are **not going to spend months learning Python syntax in isolation.** Python, GDSFactory, Lumerical, KLayout, Git, and photonics will increasingly be learned together through the project.

For now, we should still finish the small Chapter 01 repository cleanup and Git checkpoint. Then I recommend opening the new chat **“02 — Components, Ports & Reusable Building Blocks”** and starting there.